IF

EXPLOITATION · VERSION 1.1

ImmoFlow AI
Guide de déploiement

Installer la plateforme en local, la mettre en ligne sur un serveur, l'exploiter au quotidien et résoudre les incidents courants.

2
ENVIRONNEMENTS

4
PARTIES

30 min
MISE EN LIGNE

14
POINTS DE CONTRÔLE

Docker · Nginx · Let's Encrypt · PostgreSQL · Redis

Sommaire

01 **Déploiement et exploitation** 3

- Partie 1 — Installation locale 3
- Partie 2 — Mise en ligne 6
- Partie 3 — Exploitation 11
- Partie 4 — Résolution des incidents courants 12
- Checklist avant ouverture au public 13

02 **Référence d'exploitation** 14

- 1. Prérequis serveur 14
- 2. Déploiement Docker Compose 14
- 3. Reverse proxy 17
- 4. Certificat TLS 18
- 5. Durcissement 18
- 6. Stockage des documents 19
- 7. Sauvegardes 19
- 8. Mise à jour de l'application 20
- 9. Supervision 20
- 10. Montée en charge 21

GUIDE COMPLET

Déploiement et exploitation

Du poste de travail à la mise en ligne, avec les points de vigilance repérés sur le terrain.

Ce document couvre les deux situations : faire tourner la plateforme sur un poste de travail, et la mettre en ligne sur un serveur accessible depuis Internet.

Il s'applique aux deux archives :

ARCHIVE	CONTENU
`immoflow-ai-api.zip`	API FastAPI, base de données, script de peuplement
`immoflow-ai-frontend.zip`	Interface React

Les deux se décompressent directement dans le dossier courant, sans sous-dossier intermédiaire.

Partie 1 — Installation locale

1.1 Ce qu'il vous faut

OUTIL	VERSION	VÉRIFIER
Docker + Docker Compose	24+	`docker --version`
Node.js	18 minimum, 20 ou 22 recommandé	`node -v`
npm	9+	`npm -v`

Docker suffit pour le backend. Sans Docker, il vous faudra en plus Python 3.12 et PostgreSQL 16 (voir 1.5).

1.2 Backend

```
mkdir immoflow-api && cd immoflow-api
unzip ../immoflow-ai-api.zip

cp backend/.env.example backend/.env
```

Ouvrez `backend/.env` et renseignez au minimum la clé de signature :

```
# Génère une clé aléatoire
openssl rand -hex 32
```

```
SECRET_KEY=<la valeur générée>
FRONTEND_BASE_URL=http://localhost:5173
```

> **`FRONTEND_BASE_URL` est le réglage le plus souvent oublié.** C'est l'adresse utilisée pour construire les liens d'invitation et de réinitialisation de mot de passe. Si elle pointe vers un port où rien n'écoute, les liens envoyés aboutissent sur une page introuvable. Elle doit correspondre à l'adresse où le frontend est réellement servi : `http://localhost:5173` en développement.

Démarrez :

```
docker compose up -d
docker compose logs -f api
```

Vérifiez :

```
curl http://localhost:8000/health
# {"status":"healthy","database":"up","redis":"up","version":"1.0.0"}
```

La documentation interactive est sur http://localhost:8000/docs

Le compte administrateur est créé automatiquement au démarrage à partir de `FIRST_SUPERADMIN_EMAIL` et `FIRST_SUPERADMIN_PASSWORD`. Par défaut :

IDENTIFIANT	MOT DE PASSE
`admin@immoflow.ai`	`ChangeMe123!`

Changez ce mot de passe dès la première connexion.

1.3 Remplir la base

Une base vide donne une interface vide. Peuplez-la :

```
docker compose exec api python scripts/seed_data.py
```

Vous obtenez 4 agences, 41 comptes, 30 biens photographiés, 45 visites, des transactions, des documents, des conversations et un journal d'audit — avec un compte par rôle, mot de passe `Demo2026!`.

Pour un volume plus important :

```
docker compose exec api python scripts/seed_data.py --reset --properties 200 --clients 40
```

1.4 Frontend

```
mkdir immoflow-front && cd immoflow-front
unzip ../immoflow-ai-frontend.zip

npm install
npm run dev
```

L'interface est sur http://localhost:5173

Le fichier `.env` est fourni et déjà configuré : les appels `/api/*` sont relayés vers `http://localhost:8000` par le proxy de développement, sans configuration CORS.

1.5 Sans Docker

Installation manuelle du backend

```
cd backend
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# PostgreSQL doit tourner et la base exister
createdb immoflow

cp .env.example .env      # renseignez SECRET_KEY et POSTGRES_*
alembic upgrade head
PYTHONPATH=. python scripts/seed_data.py

uvicorn app.main:app --reload --port 8000
```

Redis est optionnel : sans lui, la messagerie fonctionne en REST, sans diffusion instantanée. L'API le signale au démarrage et continue de fonctionner.

1.6 Vérifier que tout est branché

```
# 1. L'API répond
curl http://localhost:8000/health

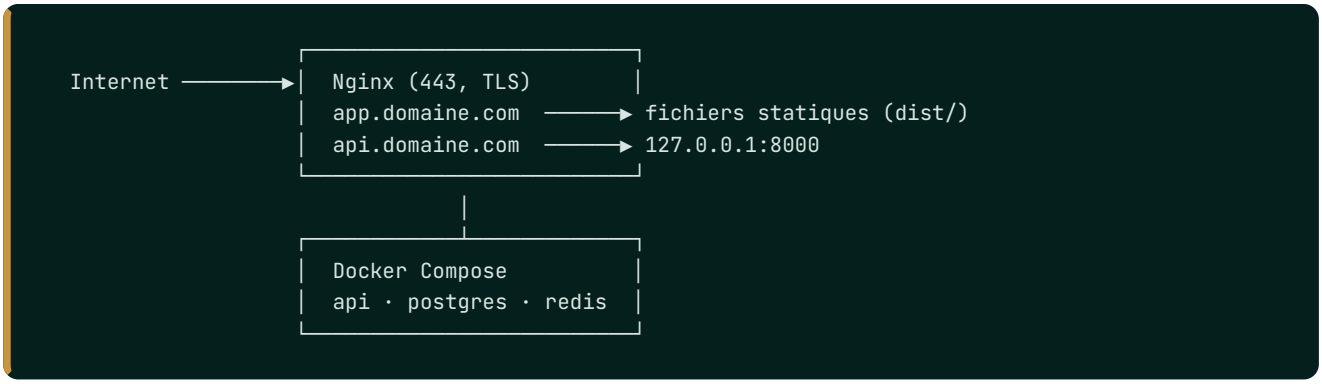
# 2. La connexion fonctionne à travers le proxy du frontend
curl -X POST http://localhost:5173/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@immoflow.ai","password":"ChangeMe123!"}'

# 3. Le catalogue public renvoie des biens
curl "http://localhost:5173/api/v1/public/properties?size=3"
```

Si les trois répondent, l'installation est correcte.

Partie 2 — Mise en ligne

2.1 Architecture cible



Deux sous-domaines, un seul serveur. Le frontend est un site statique : il n'a besoin d'aucun runtime.

2.2 Serveur

RESSOURCE	MINIMUM	CONFORTABLE
vCPU	2	4
RAM	2 Go	4 Go
Disque	20 Go	50 Go (documents déposés)
OS	Ubuntu 22.04 / Debian 12	idem

```
sudo apt update && sudo apt install -y docker.io docker-compose-plugin nginx certbot python3-certbot-nginx
sudo ufw allow 22,80,443/tcp && sudo ufw enable
```

2.3 Backend en production

```
sudo mkdir -p /opt/immoflow/api && cd /opt/immoflow/api
sudo unzip ~/immoflow-ai-api.zip
sudo cp backend/.env.example backend/.env
sudo chmod 600 backend/.env
```

`backend/.env` :

```
ENVIRONMENT=production
DEBUG=false

SECRET_KEY=<openssl rand -hex 32>

POSTGRES_HOST=db
POSTGRES_USER=immoflow
POSTGRES_PASSWORD=<mot de passe fort>
POSTGRES_DB=immoflow

REDIS_HOST=redis
REDIS_PASSWORD=<mot de passe fort>

# Restreindre aux domaines réels
BACKEND_CORS_ORIGINS=["https://app.votre-domaine.com"]
ALLOWED_HOSTS=["api.votre-domaine.com"]

PUBLIC_BASE_URL=https://api.votre-domaine.com
FRONTEND_BASE_URL=https://app.votre-domaine.com

FIRST_SUPERADMIN_EMAIL=admin@votre-domaine.com
FIRST_SUPERADMIN_PASSWORD=<mot de passe fort et temporaire>

# Envoi des invitations et réinitialisations
SMTP_ENABLED=true
SMTP_HOST=smtp.votre-fournisseur.com
SMTP_PORT=587
SMTP_USER=<identifiant>
SMTP_PASSWORD=<mot de passe>
EMAIL_FROM=no-reply@votre-domaine.com

# Assistance IA — facultatif, voir 2.8
AI_PROVIDER=none
```

Ne publiez pas les ports de PostgreSQL et Redis. Commentez leurs sections `ports` dans `docker-compose.yml` : les services communiquent par le réseau interne Docker.

```
sudo docker compose up -d --build
sudo docker compose logs -f api
```

2.4 Frontend en production

Compilez sur votre machine ou sur le serveur :

```
mkdir /tmp/front && cd /tmp/front
unzip ~/immoflow-ai-frontend.zip
```

`.env` :

```
VITE_API_URL=https://api.votre-domaine.com/api/v1
VITE_WS_URL=wss://api.votre-domaine.com/ws
VITE_DEMO_MODE=false
VITE_APP_NAME=ImmoFlow AI
```

```
npm ci
npm run build
sudo mkdir -p /var/www/immoflow
sudo cp -r dist/* /var/www/immoflow/
```

> Les variables `VITE_*` sont lues **à la compilation**. Modifier `.env` après le build n'a aucun effet : il faut recompiler.

2.5 Nginx

`/etc/nginx/sites-available/immoflow` :

```
# ---------------------------- Frontend ----------------------------
server {
    listen 443 ssl http2;
    server_name app.votre-domaine.com;

    ssl_certificate     /etc/letsencrypt/live/app.votre-domaine.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/app.votre-domaine.com/privkey.pem;

    root /var/www/immoflow;
    index index.html;

    # Routage par historique : toute route inconnue renvoie index.html,
    # sinon un rafraîchissement sur /agence/biens donnerait une 404.
    location / {
        try_files $uri $uri/ /index.html;
    }

    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}

# ------------------------------ API ------------------------------
upstream immoflow_api {
    server 127.0.0.1:8000;
    keepalive 32;
}

server {
    listen 443 ssl http2;
    server_name api.votre-domaine.com;

    ssl_certificate     /etc/letsencrypt/live/api.votre-domaine.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.votre-domaine.com/privkey.pem;

    client_max_body_size 30M;    # cohérent avec MAX_UPLOAD_SIZE_MB

    location / {
        proxy_pass http://immoflow_api;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # Messagerie temps réel : sans ces deux en-têtes, le WebSocket échoue
    # silencieusement et la messagerie retombe en REST seul.
    location /ws {
        proxy_pass http://immoflow_api;
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host       $host;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }
}

# ---------------------- Redirection HTTP ----------------------
```

```
server {
    listen 80;
    server_name app.votre-domaine.com api.votre-domaine.com;
    return 301 https://$host$request_uri;
}
```

```
sudo ln -s /etc/nginx/sites-available/immoflow /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

2.6 Certificats TLS

```
sudo certbot --nginx -d app.votre-domaine.com -d api.votre-domaine.com
sudo systemctl enable --now certbot.timer
```

En production, l'API ajoute automatiquement l'en-tête `Strict-Transport-Security: max-age=31536000; includeSubDomains`.

2.7 Première connexion

```
curl https://api.votre-domaine.com/health
```

Ouvrez `https://app.votre-domaine.com/connexion`, connectez-vous avec le compte administrateur, **changez immédiatement son mot de passe** depuis Paramètres → Sécurité.

2.8 Assistance IA

Les quatre fonctions d'assistance (recherche en langage courant, estimation, rédaction d'annonce, contrôle qualité) fonctionnent **sans aucune clé** : elles s'appuient sur les biens réellement enregistrés et sur une rédaction locale.

Pour les enrichir avec un modèle de langage :

```
AI_PROVIDER=openai        # ou groq, anthropic
AI_API_KEY=<votre clé>
AI_MODEL=gpt-4o-mini
```

Si l'appel échoue (réseau, quota, réponse invalide), l'API bascule silencieusement sur le mode local. Aucune fonctionnalité ne devient indisponible à cause d'un service tiers.

Partie 3 — Exploitation

3.1 Sauvegardes

```
sudo tee /opt/immoflow/backup.sh > /dev/null <<'SCRIPT'
#!/usr/bin/env bash
set -euo pipefail
DIR=/var/backups/immoflow
STAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p "$DIR"

docker compose -f /opt/immoflow/api/docker-compose.yml exec -T db \
  pg_dump -U immoflow immoflow | gzip > "$DIR/db_$STAMP.sql.gz"

docker run --rm -v api_uploads_data:/data -v "$DIR":/backup alpine \
  tar czf "/backup/uploads_$STAMP.tar.gz" -C /data .

find "$DIR" -name "*.gz" -mtime +30 -delete
SCRIPT
sudo chmod +x /opt/immoflow/backup.sh
echo "0 2 * * * /opt/immoflow/backup.sh >> /var/log/immoflow-backup.log 2>&1" | sudo crontab -
```

Restauration :

```
gunzip -c db_20260803_020000.sql.gz | \
  docker compose exec -T db psql -U immoflow -d immoflow
```

> Testez la restauration au moins une fois par trimestre. Une sauvegarde jamais restaurée n'est pas une sauvegarde.

3.2 Mise à jour

```
cd /opt/immoflow/api
docker compose exec -T db pg_dump -U immoflow immoflow | gzip > /var/backups/immoflow/
pre_update.sql.gz
sudo unzip -o ~/immoflow-ai-api.zip
sudo docker compose up -d --build      # les migrations s'appliquent au démarrage
curl https://api.votre-domaine.com/health
```

Frontend :

```
cd /tmp/front && unzip -o ~/immoflow-ai-frontend.zip
npm ci && npm run build
sudo rsync -a --delete dist/ /var/www/immoflow/
```

3.3 Supervision

SONDE	USAGE
`GET /health/live`	Vivacité — redémarrer le conteneur si échec
`GET /health/ready`	Disponibilité — retirer du pool si la base est injoignable
`GET /health`	Diagnostic complet

En production, les journaux sortent en JSON sur la sortie standard, avec un `request_id` renvoyé au client dans l'en-tête `X-Request-ID`. Un utilisateur qui signale une erreur peut donner cet identifiant : vous retrouvez la trace exacte.

```
docker compose logs -f api | grep '"level": "ERROR"'
```

3.4 Montée en charge

L'API est sans état : les sessions vivent dans les JWT, la diffusion temps réel passe par Redis. Plusieurs instances peuvent tourner derrière un répartiteur sans affinité de session.

Trois points d'attention en multi-instances :

- une seule instance doit appliquer les migrations ;
- le stockage des documents doit être partagé (objet plutôt que volume local) ;
- Redis devient une dépendance critique et non plus optionnelle.

Partie 4 — Résolution des incidents courants

Le lien d'invitation aboutit sur une page introuvable `FRONTEND_BASE_URL` ne correspond pas à l'adresse où le frontend est servi. Corrigez-la et redémarrez l'API. L'interface répare aussi ce cas d'elle-même : elle réécrit l'hôte du lien sur son origine réelle avant de l'afficher.

Aucun email n'est reçu Vérifiez `SMTP_ENABLED=true` et les identifiants. Sans SMTP, l'interface affiche le lien d'invitation directement dans une fenêtre, avec un bouton pour le copier ou l'ouvrir dans votre messagerie — le parcours reste praticable.

« L'API est injoignable » sur chaque écran Le backend n'est pas démarré ou `VITE_API_URL` est incorrecte. Testez `curl https://api.votre-domaine.com/health`.

Écran blanc après le build Nginx ne renvoie pas `index.html` sur les routes internes. Vérifiez la directive `try_files`.

La messagerie ne se met pas à jour instantanément Le WebSocket ne passe pas le proxy. Vérifiez le bloc `location /ws` et la présence des en-têtes `Upgrade` et `Connection`.

Aucun compte administrateur Il est créé au démarrage de l'API. Consultez les journaux : `docker compose logs api | grep "Super Admin"`. En cas de besoin : `docker compose exec api python -m app.database.init_db`.

Le catalogue est vide Comportement normal sur une base neuve. Lancez le script de peuplement, ou créez une agence puis des biens depuis l'interface.

Checklist avant ouverture au public

- [ ] `SECRET_KEY` aléatoire, unique, hors du dépôt
- [ ] `ENVIRONMENT=production`, `DEBUG=false`
- [ ] Mot de passe du Super Admin changé
- [ ] `BACKEND_CORS_ORIGINS` et `ALLOWED_HOSTS` restreints aux domaines réels
- [ ] `FRONTEND_BASE_URL` pointant vers l'adresse publique du frontend
- [ ] HTTPS actif, renouvellement automatique vérifié
- [ ] Ports PostgreSQL et Redis non publiés, mot de passe Redis défini
- [ ] `client_max_body_size` cohérent avec `MAX_UPLOAD_SIZE_MB`
- [ ] WebSocket testé à travers le proxy
- [ ] SMTP testé (invitation et réinitialisation)
- [ ] Sauvegardes automatisées **et restauration testée**
- [ ] Sondes `/health/live` et `/health/ready` branchées
- [ ] Pare-feu configuré, accès SSH par clé uniquement

ANNEXE

Référence d'exploitation

Durcissement, sauvegardes, supervision et montée en charge de l'API.

1. Prérequis serveur

RESSOURCE	MINIMUM	RECOMMANDÉ
CPU	2 vCPU	4 vCPU
RAM	2 Go	4 Go
Disque	20 Go	50 Go+ (documents)
OS	Linux (Ubuntu 22.04+, Debian 12+)	idem
Logiciels	Docker 24+, Docker Compose v2	+ reverse proxy

2. Déploiement Docker Compose

2.1 Préparation

```
git clone <repo> /opt/immoflow
cd /opt/immoflow
cp backend/.env.example backend/.env
```

2.2 Configuration de production

Éditez `backend/.env` :

```
ENVIRONMENT=production
DEBUG=false

# Clé générée : openssl rand -hex 32
SECRET_KEY=<64 caractères hexadécimaux>

# Restreindre aux domaines réels
BACKEND_CORS_ORIGINS=["https://app.votre-domaine.com"]
ALLOWED_HOSTS=["api.votre-domaine.com"]

POSTGRES_HOST=db
POSTGRES_USER=immoflow
POSTGRES_PASSWORD=<mot de passe fort>
POSTGRES_DB=immoflow

REDIS_HOST=redis
REDIS_PASSWORD=<mot de passe fort>

PUBLIC_BASE_URL=https://api.votre-domaine.com
FRONTEND_BASE_URL=https://app.votre-domaine.com

SMTP_ENABLED=true
SMTP_HOST=smtp.votre-fournisseur.com
SMTP_PORT=587
SMTP_USER=<identifiant>
SMTP_PASSWORD=<mot de passe>
EMAIL_FROM=no-reply@votre-domaine.com

FIRST_SUPERADMIN_EMAIL=admin@votre-domaine.com
FIRST_SUPERADMIN_PASSWORD=<mot de passe fort et temporaire>

AI_PROVIDER=openai        # ou groq, anthropic, none
AI_API_KEY=<clé>
```

Protégez le fichier :

```
chmod 600 backend/.env
```

2.3 Démarrage

```
docker compose up --build -d
docker compose logs -f api
```

L'entrypoint attend PostgreSQL, applique `alembic upgrade head`, crée le Super Admin et les types de biens, puis démarre Uvicorn avec 4 workers.

2.4 Vérification

```
curl http://localhost:8000/health
docker compose ps          # les 3 services doivent être "healthy"
```

Connectez-vous immédiatement et changez le mot de passe du Super Admin via `POST /api/v1/auth/change-password`.

---

3. Reverse proxy

3.1 Nginx

```
upstream immoflow_api {
    server 127.0.0.1:8000;
    keepalive 32;
}

server {
    listen 443 ssl http2;
    server_name api.votre-domaine.com;

    ssl_certificate     /etc/letsencrypt/live/api.votre-domaine.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.votre-domaine.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    client_max_body_size 30M;          # cohérent avec MAX_UPLOAD_SIZE_MB

    location / {
        proxy_pass http://immoflow_api;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # WebSocket — indispensable pour la messagerie temps réel
    location /ws {
        proxy_pass http://immoflow_api;
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host       $host;
        proxy_read_timeout 3600s;      # connexions longues
        proxy_send_timeout 3600s;
    }
}

server {
    listen 80;
    server_name api.votre-domaine.com;
    return 301 https://$host$request_uri;
}
```

3.2 Traefik (labels Docker Compose)

```
  api:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.immoflow.rule=Host(`api.votre-domaine.com`)"
      - "traefik.http.routers.immoflow.tls.certresolver=letsencrypt"
      - "traefik.http.services.immoflow.loadbalancer.server.port=8000"
```

> Le transfert des en-têtes `Upgrade` et `Connection` est **obligatoire** : sans lui, le WebSocket échoue et la messagerie retombe en mode REST uniquement.

---

4. Certificat TLS

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d api.votre-domaine.com
sudo systemctl enable --now certbot.timer     # renouvellement automatique
```

En production, l'API ajoute automatiquement l'en-tête `Strict-Transport-Security: max-age=31536000; includeSubDomains`.

---

5. Durcissement

5.1 Réseau

Ne publiez pas PostgreSQL ni Redis sur l'hôte. Supprimez leurs sections `ports` du `docker-compose.yml` en production : les services communiquent par le réseau interne Docker.

```
db:
  # ports:           ← à commenter en production
  #   - "5432:5432"
```

5.2 Pare-feu

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

5.3 Redis

Activez un mot de passe (`REDIS_PASSWORD`) et adaptez la commande :

```
redis:
  command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
```

5.4 Secrets

Ne versionnez jamais `.env`. En environnement orchestré, préférez Docker Secrets, AWS Secrets Manager, HashiCorp Vault ou l'équivalent de votre plateforme.

---

6. Stockage des documents

Volume Docker (par défaut) — le volume `uploads_data` doit être sauvegardé au même titre que la base.

Objet (recommandé à l'échelle) — implémentez le protocole `StorageBackend` dans `app/services/storage.py` (méthodes `save`, `read`, `delete`, `public_url`), puis positionnez `STORAGE_BACKEND=s3`. Aucune autre couche n'est à modifier.

---

7. Sauvegardes

7.1 Base de données

```
#!/usr/bin/env bash
# /opt/immoflow/scripts/backup.sh
set -euo pipefail
BACKUP_DIR=/var/backups/immoflow
STAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p "$BACKUP_DIR"

docker compose -f /opt/immoflow/docker-compose.yml exec -T db \
    pg_dump -U immoflow immoflow | gzip > "$BACKUP_DIR/db_$STAMP.sql.gz"

docker run --rm -v immoflow_uploads_data:/data -v "$BACKUP_DIR":/backup alpine \
    tar czf "/backup/uploads_$STAMP.tar.gz" -C /data .

find "$BACKUP_DIR" -name "*.gz" -mtime +30 -delete
```

```
0 2 * * * /opt/immoflow/scripts/backup.sh >> /var/log/immoflow-backup.log 2>&1
```

7.2 Restauration

```
gunzip -c db_20260731_020000.sql.gz | \
  docker compose exec -T db psql -U immoflow -d immoflow
```

> Testez la restauration au moins une fois par trimestre. Une sauvegarde jamais restaurée n'est pas une sauvegarde.

8. Mise à jour de l'application

```
cd /opt/immoflow
docker compose exec -T db pg_dump -U immoflow immoflow | gzip > /var/backups/immoflow/
pre_update.sql.gz
git pull
docker compose up --build -d      # migrations appliquées automatiquement
docker compose logs -f api
curl https://api.votre-domaine.com/health
```

Retour arrière

```
git checkout <tag précédent>
docker compose up --build -d
cd backend && alembic downgrade -1   # uniquement si une migration doit être annulée
```

9. Supervision

9.1 Sondes

SONDE	USAGE
`GET /health/live`	Vivacité — redémarrage du conteneur si échec
`GET /health/ready`	Disponibilité — retrait du pool si la base est injoignable
`GET /health`	Diagnostic complet (base + Redis + version)

Exemple Kubernetes :

```
livenessProbe:
  httpGet: { path: /health/live, port: 8000 }
  initialDelaySeconds: 20
  periodSeconds: 30
readinessProbe:
  httpGet: { path: /health/ready, port: 8000 }
  initialDelaySeconds: 10
  periodSeconds: 10
```

9.2 Journaux

En production, les logs sont émis en **JSON** sur la sortie standard, prêts pour Loki, CloudWatch, Datadog ou l'ELK. Chaque ligne d'accès contient méthode, chemin, code, durée et `request_id` — le même identifiant est renvoyé au

client dans l'en-tête `X-Request-ID` et dans les réponses d'erreur, ce qui permet de relier un ticket utilisateur à sa trace serveur.

```
docker compose logs -f api | grep '"level": "ERROR"'
```

9.3 Indicateurs à surveiller

- Taux de réponses 5xx
- Latence (en-tête `X-Process-Time-ms`)
- Réponses 429 (saturation du rate limiting)
- Connexions PostgreSQL actives par rapport à `DB_POOL_SIZE`
- Mémoire Redis et taille du canal Pub/Sub
- Espace disque du volume de documents

---

10. Montée en charge

10.1 Vertical

Ajustez le nombre de workers Uvicorn — règle usuelle : `(2 × vCPU) + 1`.

```
command: ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000",
          "--workers", "9", "--proxy-headers"]
```

Alignez le pool de connexions : `DB_POOL_SIZE × nombre de workers` doit rester en deçà de `max_connections` de PostgreSQL.

10.2 Horizontal

L'API est **sans état** : les sessions vivent dans les JWT, la diffusion temps réel passe par Redis Pub/Sub. Plusieurs instances peuvent donc tourner derrière un répartiteur de charge sans affinité de session — un utilisateur connecté à l'instance A reçoit bien les messages émis depuis l'instance C.

Points d'attention en multi-instances :

- une seule instance doit appliquer les migrations (job d'initialisation dédié) ;
- le stockage des documents doit être partagé (objet plutôt que volume local) ;
- Redis devient une dépendance critique et non plus optionnelle.

10.3 Base de données

- Réplicas en lecture pour les rapports et le catalogue public.
- PgBouncer en mode transaction si le nombre d'instances augmente fortement.

- Surveillance des index : ceux prévus couvrent les filtres les plus fréquents (`agency_id + status` , `agent_id + status` , `conversation_id + created_at` , `user_id + is_read`).

---

11. Tâches planifiées

Le worker Celery est optionnel mais recommandé en production :

```
docker compose --profile workers up -d
```

Purge des jetons expirés (à planifier quotidiennement) :

```
docker compose exec worker celery -A app.worker.celery_app call tokens.purge_expired
```

---

12. Checklist de mise en production

- [ ] `SECRET_KEY` aléatoire, unique, stocké hors du dépôt
- [ ] `ENVIRONMENT=production` , `DEBUG=false`
- [ ] Mot de passe du Super Admin changé après le premier démarrage
- [ ] `BACKEND_CORS_ORIGINS` et `ALLOWED_HOSTS` restreints
- [ ] HTTPS actif, renouvellement automatique du certificat
- [ ] Ports PostgreSQL et Redis non publiés, mot de passe Redis défini
- [ ] `client_max_body_size` du proxy cohérent avec `MAX_UPLOAD_SIZE_MB`
- [ ] WebSocket testé à travers le proxy
- [ ] Sauvegardes automatisées **et restauration testée**
- [ ] Journaux JSON collectés, alertes sur les 5xx
- [ ] Sondes `/health/live` et `/health/ready` branchées
- [ ] SMTP fonctionnel (test d'invitation et de réinitialisation)
- [ ] Purge des jetons planifiée
- [ ] Pare-feu configuré, accès SSH par clé uniquement